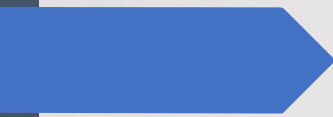




BSM422 Bilgisayarla Görme ve Görüntüleme Teknikleri

Dr. Öğr. Üyesi Mehmet Zahid YILDIRIM



Bu ders;

- MNIST dataseti ile sınıflandırma
- Sıfırdan CNN modeli oluşturma
 - Conv2D,
 - Dense,
 - Dropout,
 - Flatten,
 - MaxPooling2D
- Oluşturulan modeli kaydetme ve tahmin işlemleri

Veri Setini Yükleme

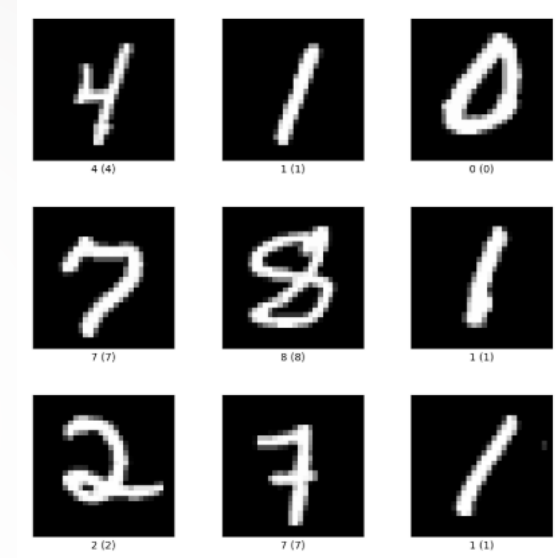
MNIST

MNIST:

- El yazısı rakam görüntülerinden oluşur
- 0–9 arasında 10 sınıf vardır
- Görüntüler:
 - 28x28 boyutunda
 - gri seviye (siyah-beyaz)

Veri seti:

- 60.000 eğitim görüntüsü
 - 10.000 test görüntüsü
- içerir.



```
[1]: # library
      from keras.datasets import mnist
```

Train ve Test Kümesi Oluşturma

```
[2]: # train-test split
(train_images, train_labels) = mnist.load_data()[0]
(test_images, test_labels) = mnist.load_data()[1]

# (train_images, train_labels), (test_images, test_labels) = mnist.load_data()

print("Shape of Train Images: ", train_images.shape)
print("Shape of Test Images : ", test_images.shape)

Shape of Train Images: (60000, 28, 28)
Shape of Test Images : (10000, 28, 28)
```

Verileri Görüntüleme

- Bir MNIST görüntüsü aslında şöyle sayısal değerlerden oluşur:

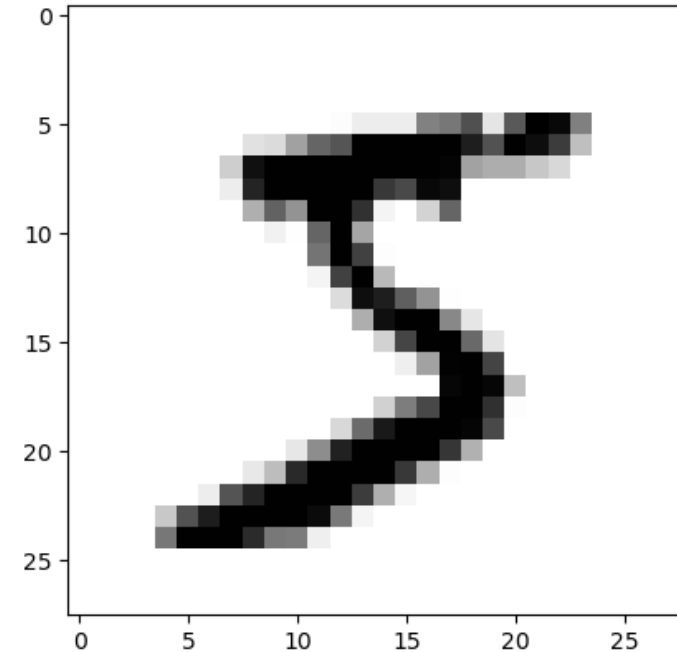
[[0 0 0 ...]

[0 0 255 ...]

[34 180 255 ...]]

```
[3]: # displaying data
import matplotlib.pyplot as plt

plt.imshow(train_images[0], cmap="gray_r")
plt.show()
```



Verileri Hazırlama

- ▶ CNN katmanları görüntüleri şu formatta bekler: **(samples, height, width, channels)**

Buradaki:

- samples → görüntü sayısı
- height → yükseklik
- width → genişlik
- channels → renk kanalı

```
[4]: # preparing data
# 3D --> 4D
train_images = train_images.reshape(train_images.shape[0], 28, 28, 1)
test_images = test_images.reshape(test_images.shape[0], 28, 28, 1)
print("Shape of Train Images: ", train_images.shape)
print("Shape of Test Images : ", test_images.shape)

# ./255
train_images = train_images.astype("float32")
test_images = test_images.astype("float32")

train_images /= 255
test_images /= 255

Shape of Train Images: (60000, 28, 28, 1)
Shape of Test Images : (10000, 28, 28, 1)
```

Model Oluşturma

- **Keras Sequential API:** Bu yapı katmanların sırayla birbirine bağlandığı model tipidir.

Katman1 → Katman2 → Katman3

```
[5]: # creating model
from keras.models import Sequential
from keras.layers import Input, Conv2D, Dense, Dropout, Flatten, MaxPooling2D

model = Sequential([
    Input(shape=(28,28,1)),
    Conv2D(28, kernel_size=(3,3), activation="relu"),
    MaxPooling2D(pool_size=(2,2)),
    Flatten(),
    Dense(128, activation="relu"),
    Dropout(0.2),
    Dense(10, activation="softmax")
])
```

Model Oluşturma

- **Conv2D:** Bu katman görüntülerden kenarlar, eğriler, şekiller ve desenler gibi özellikleri öğrenir.
- **Conv2D(28, ...):** Filtre sayısıdır. Model burada 28 farklı özellik haritası üretir.

```
[5]: # creating model
from keras.models import Sequential
from keras.layers import Input, Conv2D, Dense, Dropout, Flatten, MaxPooling2D

model = Sequential([
    Input(shape=(28,28,1)),
    Conv2D(28, kernel_size=(3,3), activation="relu"),
    MaxPooling2D(pool_size=(2,2)),
    Flatten(),
    Dense(128, activation="relu"),
    Dropout(0.2),
    Dense(10, activation="softmax")
])
```


Model Oluşturma

- **kernel_size=(3,3)**: Convolution filtresinin boyutudur.
- **activation="relu"**: $f(x) = \max(0, x) \rightarrow$ Negatif değerleri sıfırlar.

```
[5]: # creating model
from keras.models import Sequential
from keras.layers import Input, Conv2D, Dense, Dropout, Flatten, MaxPooling2D

model = Sequential([
    Input(shape=(28,28,1)),
    Conv2D(28, kernel_size=(3,3), activation="relu"),
    MaxPooling2D(pool_size=(2,2)),
    Flatten(),
    Dense(128, activation="relu"),
    Dropout(0.2),
    Dense(10, activation="softmax")
])
```

Model Oluşturma

- **MaxPooling2D(pool_size=(2,2))**: Özellik haritasını küçültür.

$26 \times 26 \rightarrow 13 \times 13$ nasıl çalışır? $\rightarrow \begin{bmatrix} 1 & 5 \\ 2 & 9 \end{bmatrix} \rightarrow 9$

```
[5]: # creating model
from keras.models import Sequential
from keras.layers import Input, Conv2D, Dense, Dropout, Flatten, MaxPooling2D

model = Sequential([
    Input(shape=(28,28,1)),
    Conv2D(28, kernel_size=(3,3), activation="relu"),
    MaxPooling2D(pool_size=(2,2)),
    Flatten(),
    Dense(128, activation="relu"),
    Dropout(0.2),
    Dense(10, activation="softmax")
])
```

Model Oluşturma

- **Flatten():** CNN çıkışını düzleştirir. Tek boyutlu vektöre çevirir.

(13, 13, 28) → 4732

```
[5]: # creating model
from keras.models import Sequential
from keras.layers import Input, Conv2D, Dense, Dropout, Flatten, MaxPooling2D

model = Sequential([
    Input(shape=(28,28,1)),
    Conv2D(28, kernel_size=(3,3), activation="relu"),
    MaxPooling2D(pool_size=(2,2)),
    Flatten(),
    Dense(128, activation="relu"),
    Dropout(0.2),
    Dense(10, activation="softmax")
])
```

Model Oluşturma

- **Dense(128, activation="relu")**: Tam bağlantılı (fully connected) katmandır. 128 nörondan oluşur.

```
[5]: # creating model
from keras.models import Sequential
from keras.layers import Input, Conv2D, Dense, Dropout, Flatten, MaxPooling2D

model = Sequential([
    Input(shape=(28,28,1)),
    Conv2D(28, kernel_size=(3,3), activation="relu"),
    MaxPooling2D(pool_size=(2,2)),
    Flatten(),
    Dense(128, activation="relu"),
    Dropout(0.2),
    Dense(10, activation="softmax")
])
```

Model Oluşturma

- **Dropout(0.2):** Overfitting azaltmak için kullanılır. Eğitim sırasında %20 oranında nöron rasgele kapatılır.

```
[5]: # creating model
from keras.models import Sequential
from keras.layers import Input, Conv2D, Dense, Dropout, Flatten, MaxPooling2D

model = Sequential([
    Input(shape=(28,28,1)),
    Conv2D(28, kernel_size=(3,3), activation="relu"),
    MaxPooling2D(pool_size=(2,2)),
    Flatten(),
    Dense(128, activation="relu"),
    Dropout(0.2),
    Dense(10, activation="softmax")
])
```

Model Oluşturma

- **Dense(10, activation="softmax")**: MNIST veri seti 10 sınıftan oluşur. Bu sebeple çıkış katmanında 10 nöron bulunur.
- **softmax**: Çıktıları olasılığa dönüştürür.

```
[5]: # creating model
from keras.models import Sequential
from keras.layers import Input, Conv2D, Dense, Dropout, Flatten, MaxPooling2D

model = Sequential([
    Input(shape=(28,28,1)),
    Conv2D(28, kernel_size=(3,3), activation="relu"),
    MaxPooling2D(pool_size=(2,2)),
    Flatten(),
    Dense(128, activation="relu"),
    Dropout(0.2),
    Dense(10, activation="softmax")
])
```

Compiling Model

```
[6]: # compiling model
model.compile(optimizer="adam",
              loss = "sparse_categorical_crossentropy",
              metrics = ["accuracy"])
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 28)	280
max_pooling2d (MaxPooling2D)	(None, 13, 13, 28)	0
flatten (Flatten)	(None, 4732)	0
dense (Dense)	(None, 128)	605,824
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 10)	1,290

Fitting Model

```
[7]: # fitting model
history = model.fit(x = train_images,
                    y = train_labels,
                    epochs = 10)
```

```
Epoch 1/10
1875/1875 ————— 14s 7ms/step - accuracy: 0.9452 - loss: 0.1826
Epoch 2/10
1875/1875 ————— 13s 7ms/step - accuracy: 0.9806 - loss: 0.0639
Epoch 3/10
1875/1875 ————— 12s 7ms/step - accuracy: 0.9862 - loss: 0.0446
Epoch 4/10
1875/1875 ————— 12s 6ms/step - accuracy: 0.9897 - loss: 0.0334
Epoch 5/10
1875/1875 ————— 12s 7ms/step - accuracy: 0.9919 - loss: 0.0256
Epoch 6/10
1875/1875 ————— 12s 7ms/step - accuracy: 0.9936 - loss: 0.0207
Epoch 7/10
1875/1875 ————— 13s 7ms/step - accuracy: 0.9945 - loss: 0.0167
Epoch 8/10
1875/1875 ————— 12s 7ms/step - accuracy: 0.9955 - loss: 0.0134
Epoch 9/10
1875/1875 ————— 13s 7ms/step - accuracy: 0.9963 - loss: 0.0114
Epoch 10/10
1875/1875 ————— 12s 7ms/step - accuracy: 0.9963 - loss: 0.0105
```


Evaluating Model

```
[8]: # evaluating model
test_loss, test_accuracy = model.evaluate(test_images, test_labels)
print("LOSS:", test_loss)
print("ACC :", test_accuracy)

313/313 ————— 1s 3ms/step - accuracy: 0.9869 - loss: 0.0468
LOSS: 0.04679010435938835
ACC : 0.9868999719619751
```

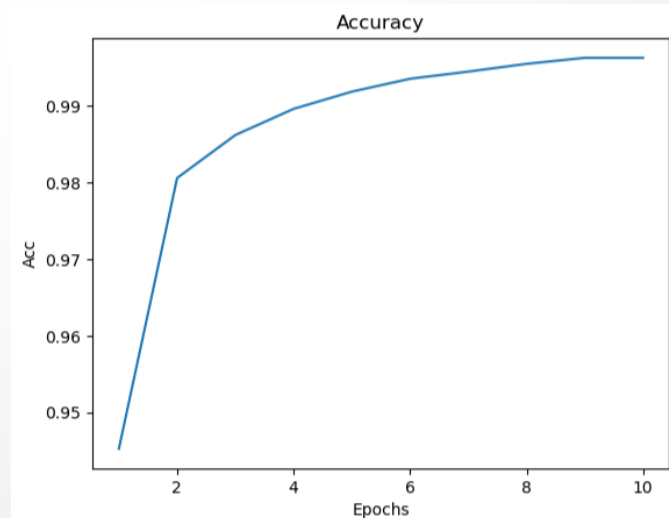
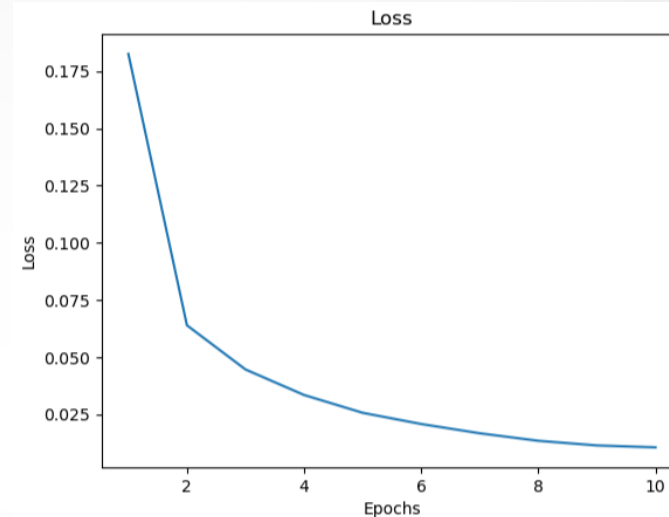
Loss ve Accuracy Grafikleri

```
[11]: # loss, acc graph
import matplotlib.pyplot as plt
history_dict = history.history

epochs = range(1,11)
loss = history_dict["loss"]
acc = history_dict["accuracy"]

plt.plot(epochs, loss)
plt.title("Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.show()

plt.plot(epochs, acc)
plt.title("Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Acc")
plt.show()
```



Modeli Kaydetme

```
[12]: # saving model  
model.save("model.keras")
```

/ mzy /			
☐ Name	Modified	Size	
☐ dataset	14 days ago		
☐ tomato_aug_comp	13 days ago		
☐ aksamGrubu.ipynb	last month	925 B	
☐ bilgisayarlaGoru.ipynb	last month	1.1 KB	
☐ ders5.ipynb	27 days ago	22.5 KB	
☐ ders6.ipynb	20 days ago	9.1 KB	
☒ ders8.ipynb	4 seconds ago	76.1 KB	
☐ model.keras	14 seconds ago	7 MB	

Modeli Yükleme ve Tahmin

```
[14]: # loading model and prediction
from keras.models import load_model
model = load_model("model.keras")

i = 7
prediction = model.predict(test_images[i].reshape(1,28,28,1))
print("Predicted Number: ", prediction.argmax())

plt.imshow(test_images[i].reshape(28,28), cmap="gray_r")
plt.show()
```

Predicted Number: 9

